

Matrix Multiplication - Sequential, Parallel and BLAS

1 Introduction

This report presents the performance and correctness comparison of sequential, parallel / threaded (using C++ threads) and BLAS matrix multiplication of double data type. The comparison is also performed between a couple of Mac processors and an Intel processor.

2 GEMM Operation

Matrix multiplication is a fundamental binary operation in linear algebra that produces a new matrix from two distinct matrices, denoted as the product $C = AB$. Unlike element-wise multiplication, this operation involves the linear combination of rows from the first matrix (A) and columns from the second matrix (B). The value of each scalar entry c_{ij} in the resulting matrix is calculated as the dot product of the i-th row of A and the j-th column of B. This is formalized by the summation:

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj}$$

3 Experimental Setup

3.1 Hardware

Parameter	Primary Machine	Secondary Machine 1	Secondary Machine 2
Laptop	MacBook Air	Lenovo Ideapad Gaming 3	MacBook Air
Manufacturer	Mac	Lenovo	Mac
CPU	Apple Silicon M1 chip (ARM-based)	AMD Ryzen 7 5800h	Apple Silicon M4 chip (ARM-based)
Clock speed	3.2 GHz (Performance Cores) 2.06 GHz (Efficiency Cores)	3.2 Ghz	4.40 GHz (Performance Cores) 2.85 GHz (Efficiency Cores)
Instructions per clock cycle	8 (approx.)	4 (approx.)	8 (approx.)
Cache Size	L1: 192KB (Instr.) + 128KB (Data) per P-core L2: 12MB (Shared P-cores) + 4MB (Shared E-cores) SLC: 8MB (System Level Cache)	L1: 64KB (32KB Instr. + 32KB Data) L2: 4MB SLC: 16 MB	L1: 192KB (Instr) + 128KB (Data) L2: 16MB (P-Core Shared) + 4MB (E-Core Shared) SLC: 8MB (System Level Cache)

Physical Cores	8 (4 Performance + 4 Efficiency)	8	10 Cores (4 Performance + 6 Efficiency)
Logical Cores	8	16	10
RAM	8GB	8GB	16GB

3.2 Software

Parameter	Primary Machine	Secondary Machine 1	Secondary Machine 2
Operating System	macOS (Apple Silicon native)	Ubuntu 22	macOS (Apple Silicon native)
Compiler	clang++	clang++	clang++
Optimization Level	-O0	-O0	-O0
Language Standard	-std=c++17	-std=c++17	-std=c++17
Linked Framework	Accelerate - Blas	Blas	Accelerate - Blas
IDE	Visual Studio Code	Visual Studio Code	Visual Studio Code

4 Implementation Approach

4.1 Sequential

Sequential matrix multiplication utilizes a three nested loop structure. The outer loops iterate through the rows of matrix A and the columns of matrix B, while the innermost loop computes the dot product of these vectors element-by-element.

4.2 Parallel / Multi-threaded

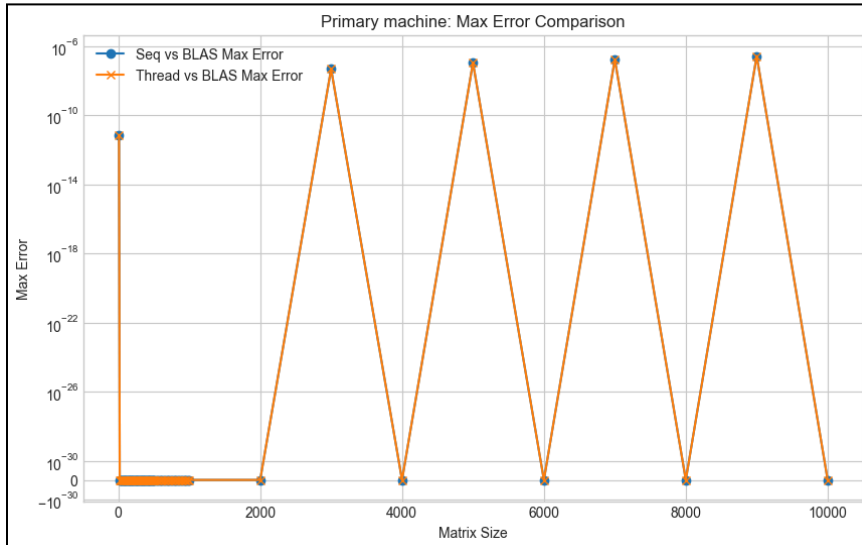
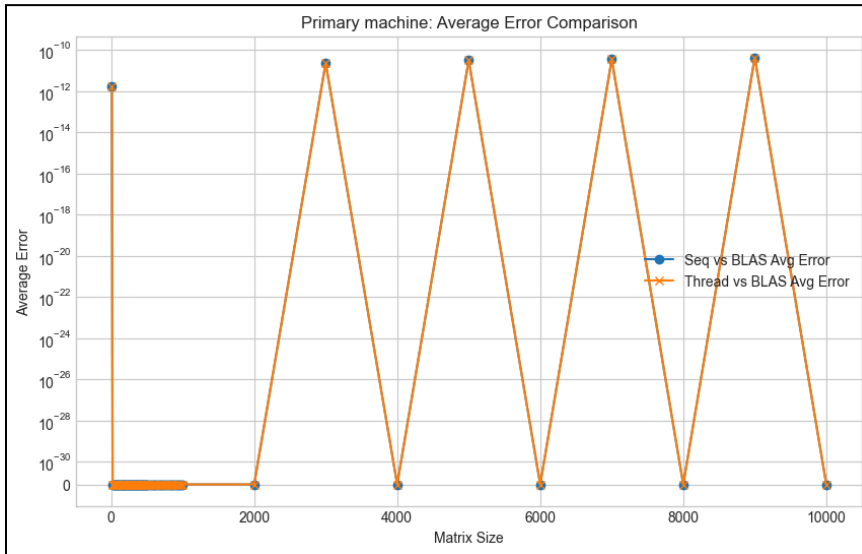
The algorithm works by breaking down the work into parts. It does this by splitting the work into rows. The rows of the new matrix C are then divided into groups. Each group is given to a thread to work on. For example if you have a computer, with 8 cores and you are working on a matrix with 1000 rows each thread will work on 125 rows of the matrix C on its own.

4.3 BLAS

Unlike high-level languages that may store matrices as arrays of arrays (which can be scattered across memory), BLAS requires matrices to be flattened into single, contiguous 1D arrays. BLAS utilises SIMD Vectorization and Cache Tiling to accelerate matrix multiplication.

5 Experiment Results

5.1 Correctness Comparison



5.1.1 Interpretation

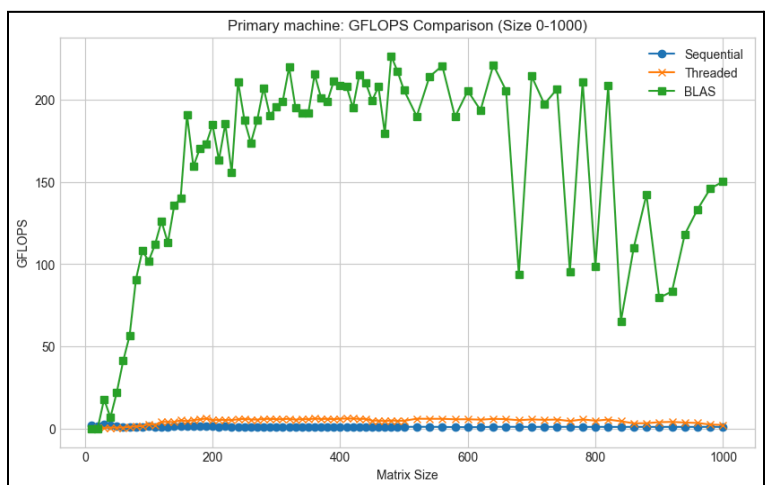
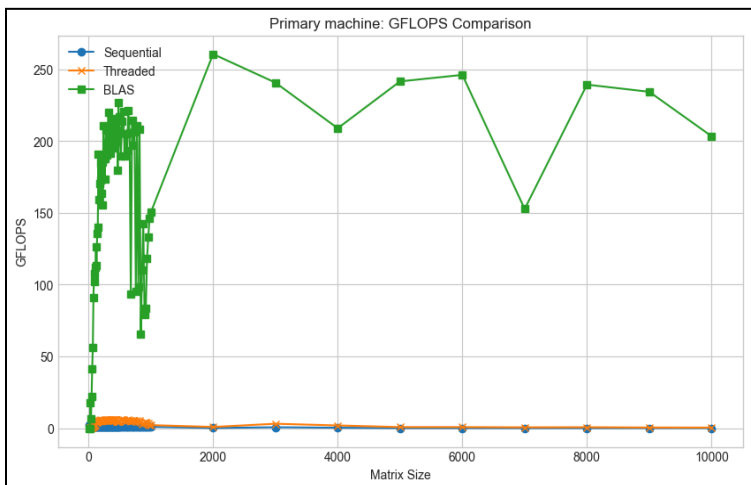
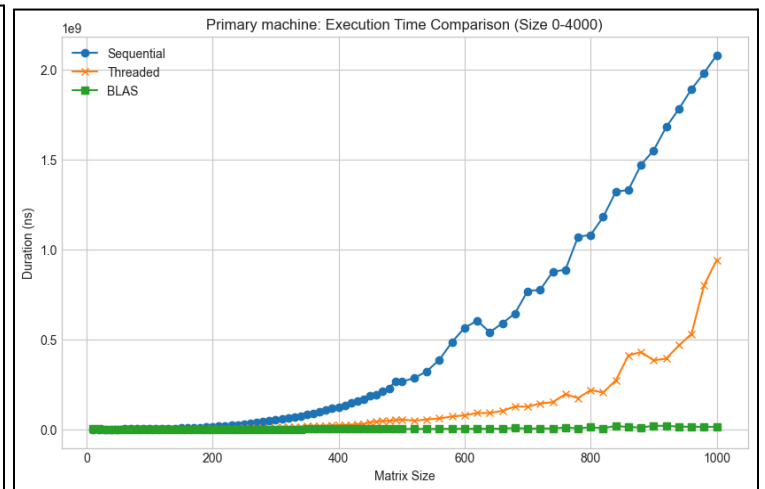
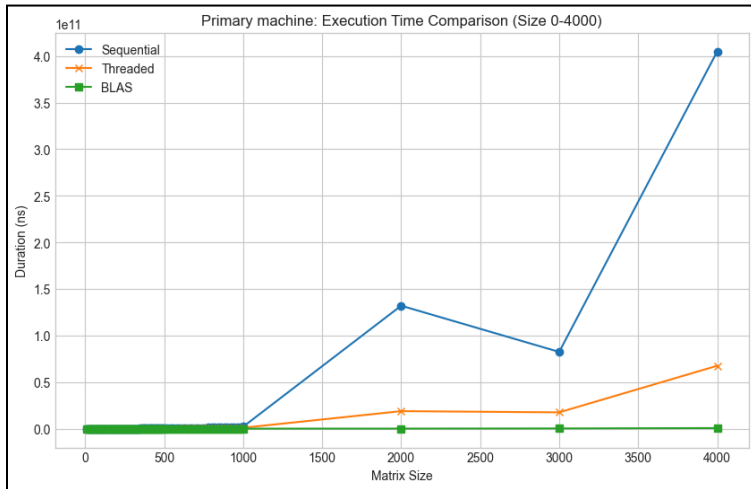
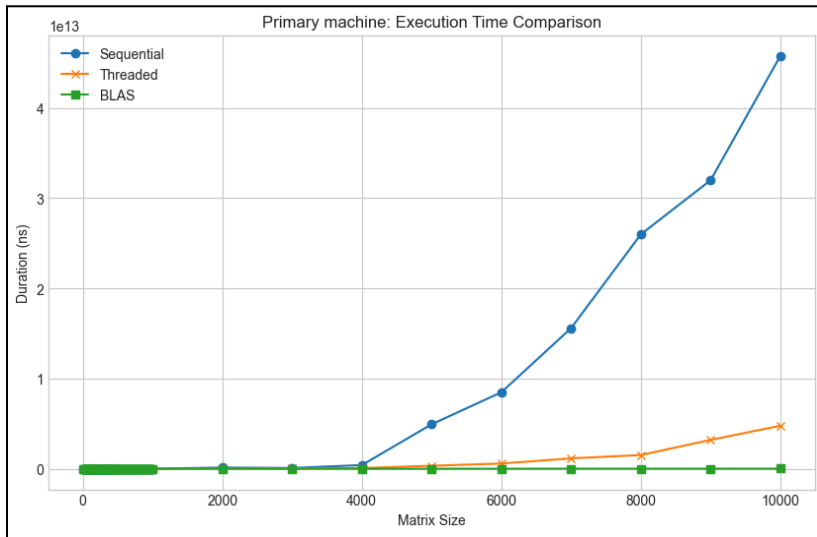
- For all the sizes of matrices, there was no error between Sequential and Threaded matrix multiplication implementations.
- However, there were few equal and negligible residual errors when they were compared with BLAS.
- The residual error between Seq vs BLAS and Threaded vs BLAS are identical.
- The max residual error peaked at the sizes 5000, 7000 and 9000.
- For larger sizes like 2000, 4000, 6000, 8000 and 10000, the residual error is near-zero suggesting similarity at bit level.

5.1.2 Analysis

- **Floating Point Non-Associativity:** In floating-point arithmetic, the order in which you add numbers changes the rounding errors attributing to the residual errors.
- **Naive Parallel:** The row decomposition threading strategy is very similar to the sequential implementation. Hence the errors between Sequential / Threaded vs BLAS are identical.

- **Random Input data:** There is a drop in residual error for alternating sizes like 2000, 4000, 6000, 8000, and 10000. It could most likely depend on the random numbers initialised for those sizes of matrices.

5.2 Performance Comparison



5.2.1 Interpretation

- **Sequential:** The execution time explodes for matrix size 4000 and shows a cubic growth with increase in matrix size. In the zoomed version of execution time comparison, the execution time has drastically increased at sizes 1000 and 500. The peak GFLOPS for sequential is only 2.25 GFLOPS for size 30.
- **Threaded:** Threaded is relatively faster than Sequential but it follows a steep curve too. The peak GFLOPS for threaded is 6 GFLOPS for size 190.
- **BLAS:** The execution time for BLAS is almost flat even in the zoomed in version of the graph. This depicts the scale of speed difference between BLAS and other methods. BLAS achieved 260 GFLOPS for size 2000.

5.2.2 Analysis

- **SIMD vs Scalar:** The significant increase of BLAS performance attributes to use of SIMD (Single Instruction, Multiple Data) via the NEON instruction set, while others use scalar instructions.
- **Cache Blocking (Memory Bound):** The significant slowness in sequential and threaded execution is because the CPU Cache (12MB L2) is insufficient to accommodate rows of large matrices; and also in sequential and threaded the operation is carried out row by row. The CPU spends hundreds of cycles waiting for data to be fetched from the RAM for every calculation. This is called being Memory Bound.
- **Cache Tiling:** In BLAS, the cache tiling technique breaks down the matrix to tiny tiles that fit them in L1 cache. It finishes all the calculations on the fetched tile before proceeding to the next; this significantly cuts down the CPU wait time for memory load.
- **Threading overhead:** The naive implementation of thread has created memory contention, where all the threads compete for the same memory bus of RAM. Hence the performance of thread is not close to BLAS despite use of all 8 threads.

6 Theoretical Peak Performance

The formula for theoretical peak performance is, $\text{GFLOPS} = \text{Clock Speed} \times \text{Cores} \times \text{FLOPs per Cycle}$. The theoretical peak performance for primary machine is calculated as,

- Clock Speed: 3.2 GHz
- Physical Cores: 4 (Performance)
- FLOPs per Cycle: 32 (4 NEON pipelines $\times$ 2 FMA ops $\times$ 4-wide vector)

Applying in the formula, $3.2 \text{ GHz} \times 4 \text{ Cores} \times 32 \text{ ops} = 409.6 \text{ GFLOPS}$. Below are peak performances of the sequential, threaded and BLAS matrix multiplication runs on the primary machine.

- **Sequential:** 2.25 GFLOPS (0.5 % of theoretical peak performance)
- **Threaded:** 6.05 GLOPS (1.4 % of theoretical peak performance)
- **BLAS:** 260.6 GFLOPS (63.6 % of theoretical peak performance)

6.1 Analysis

6.1.1 Sequential vs Theoretical peak performance

- **Scalar Execution:** The code processes one number at a time. The M1 is designed to process 4 to 8 numbers at once using NEON (SIMD) instructions. By doing it one by one, there is an immediate loss of 75-87% of potential speed.
- **Single Core:** By using 1 CPU core, leaving the other 7 idle, the CPU power is underutilized by 7 times.
- **Cache Thrashing:** Naive loops (i, j, k) access memory in inefficient patterns. The CPU spends hundreds of cycles waiting for data to travel from RAM to the registers, meaning the ALU is idle for 99% of the time.

6.1.2 Threaded vs Theoretical peak performance

- **Scalar Bottleneck:** Even with 8 threads, if each thread is still doing scalar math and not using NEON vectors, the math throughput is still low.

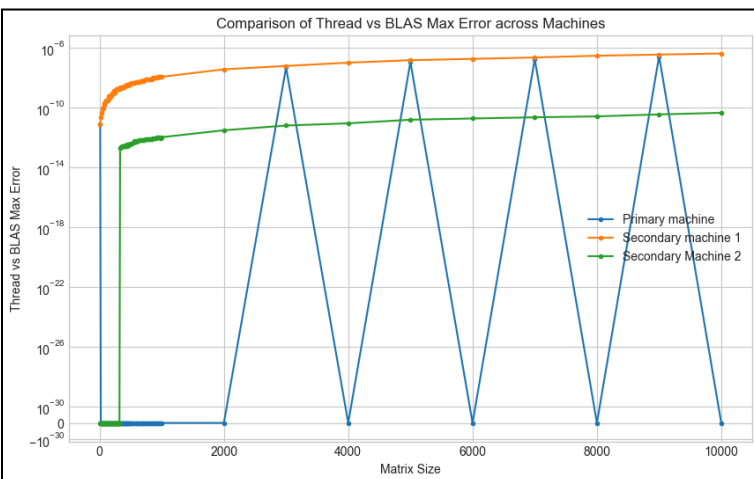
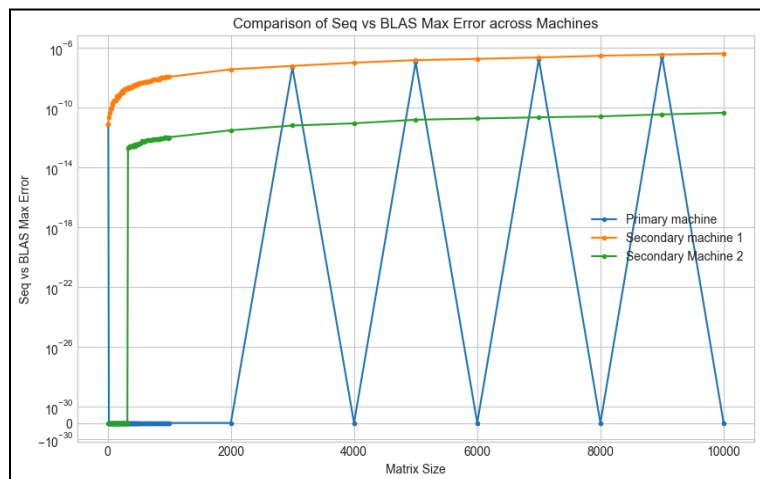
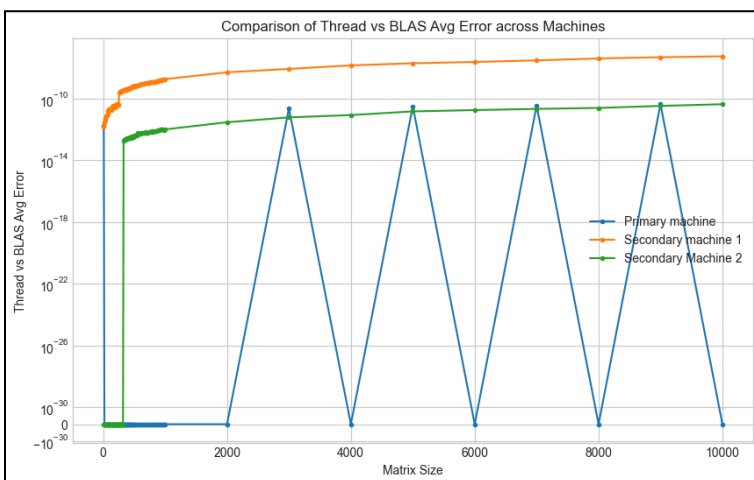
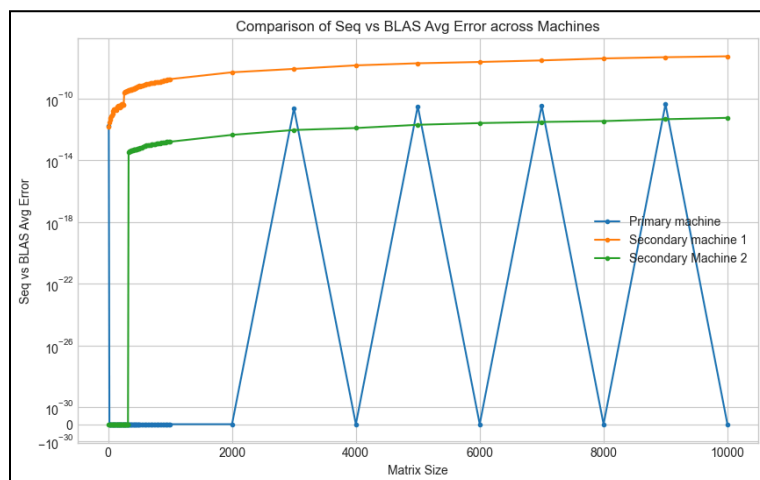
- **Memory Contention:** Without Tiling or Blocking (optimizing data to stay in the fast L1/L2 cache), threads compete with each other for access to the slower main RAM. This starves CPU cores of data.
- **False Sharing:** If the threads try to write to memory addresses that are close to each other, the CPU cores have to constantly pause to synchronize their caches which is called Cache Coherency Traffic. This kills the performance.

6.1.3 BLAS vs Theoretical peak performance

- **SIMD/Vectorization:** It uses NEON instructions to multiply 4 pairs of numbers in a single cycle.
- **Cache Tiling:** It breaks the matrix into small tiles that fit perfectly inside the M1's L1 (192KB) and L2 (12MB) caches, ensuring the CPU almost never waits for slow RAM.
- **The 36% gap in performance:**
 - **Thermal Throttling:** When all 4 Performance cores run dense vector math, the M1 lowers its clock speed from 3.2 GHz to ~3.0 GHz or lower to prevent overheating.
 - **Instruction Overhead:** Theoretical peak assumes only math operations. In reality, the chip must also calculate addresses, manage loops, and load data. These housekeeping instructions take up pipeline slots.
 - **The FP64 Limit:** The benchmark uses double (64-bit float), so the performance hit the limit of the standard CPU cores. The M1's high performance AMX Coprocessor, generally does not support 64-bit precision, so it remains inactive.

7 Processors Comparison

7.1 Correctness Comparison



7.1.1 Interpretation

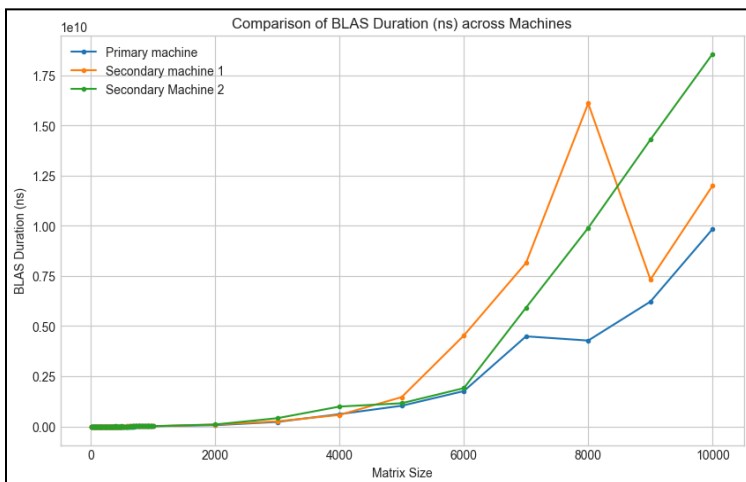
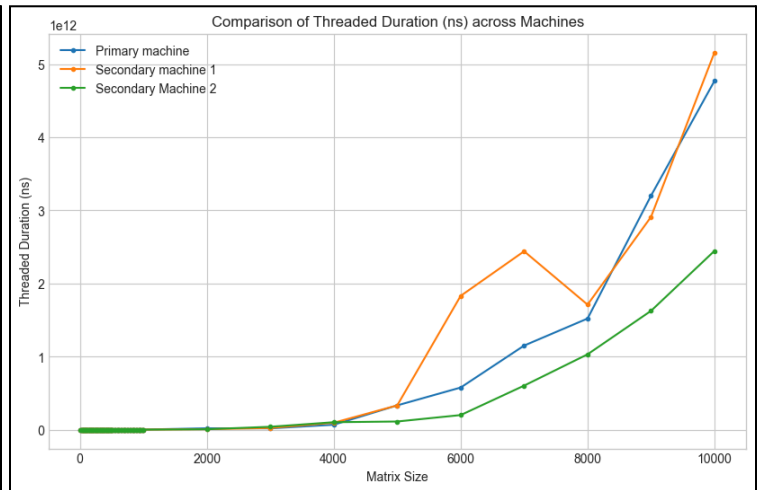
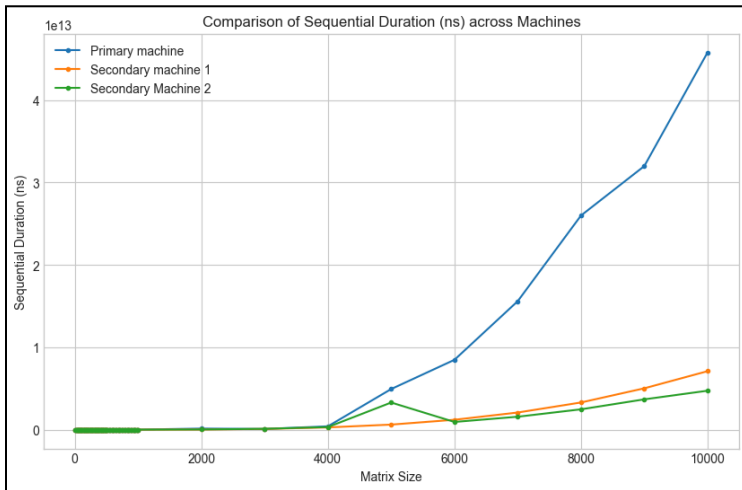
- **Primary Machine (Mac M1):** In all cases, the error rates are volatile with alternating significant highs and lows. The errors started appearing for larger size matrices only.
- **Secondary Machine 1 (Lenovo Ideapad):** In all cases of error, there is a smooth and monotonic increase in error for increasing matrix size. It shows a predictable increase.
- **Secondary Machine 2 (Mac M4):** In all cases, has the lowest error for larger matrices. However, error rates have started appearing for medium size matrices also which was not the case for Primary machine.

7.1.2 Analysis

- **Cache Alignment Artifacts:** In case of Primary machine 1, even matrix sizes likely align perfectly with the caches. Odd sizes might force remainder loops or peeling, changing the order of floating-point addition.
- **Standard Floating-Point Accumulation:** In case of Secondary Machine 1, the generic BLAS implementation follows standard x86 error growth rules. As trillions of additions occur, rounding errors accumulate linearly or via square root laws.
- **Architectural & Library Refinement:** In case of Secondary Machine 2, larger L2 Cache and improved AMX units allow for more robust blocking strategies.

7.2 Performance Comparison

7.2.1 Execution Time



Sequential Matrix Multiplication

- **Mac M1 (Primary):** Slowest of all. The blue line explodes upwards, reaching more than 4.5 times for N=10,000.
- **Lenovo (Secondary 1):** Fastest of all three. The orange line stays significantly lower, finishing well under 10 power 13 ns.
- **Mac M4 (Secondary 2):** In between the other two machines. The green line is faster than the M1 but slower than the Lenovo for sequential tasks.
- **Reason:** The Lenovo's Ryzen 5800H likely has a higher **Turbo Boost frequency** for sustained single-core loads and potentially better compiler optimization for standard x86 loops compared to the ARM-based M1/M4 in -O0 (unoptimized) mode. The M1/M4's efficiency cores might also be engaged differently by the scheduler.

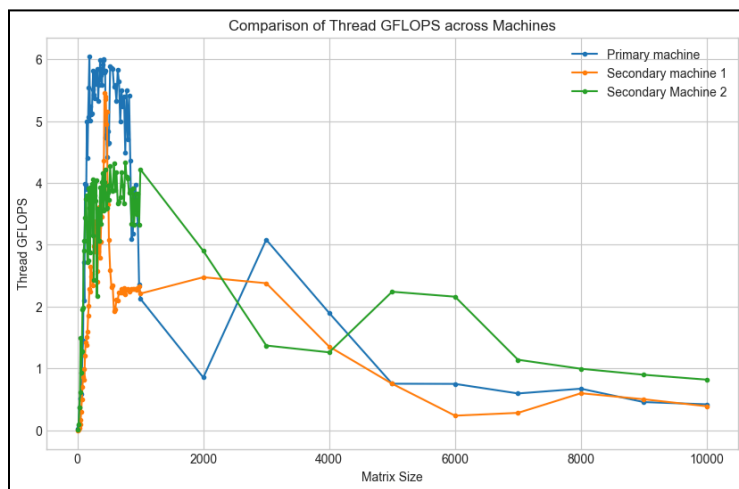
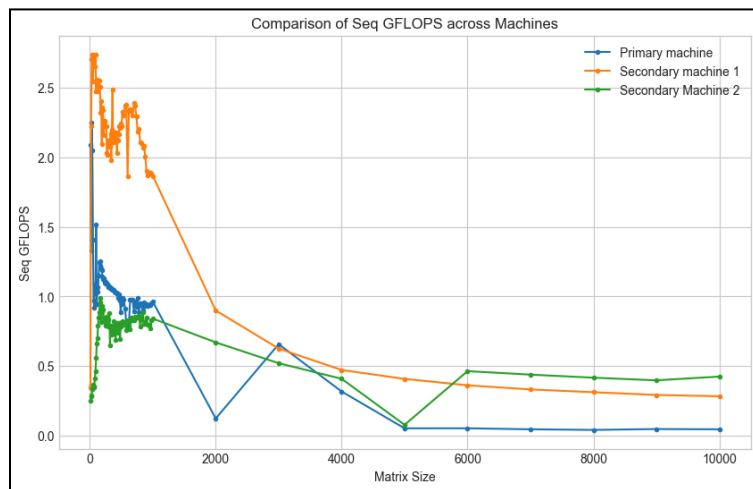
Threaded Matrix Multiplication

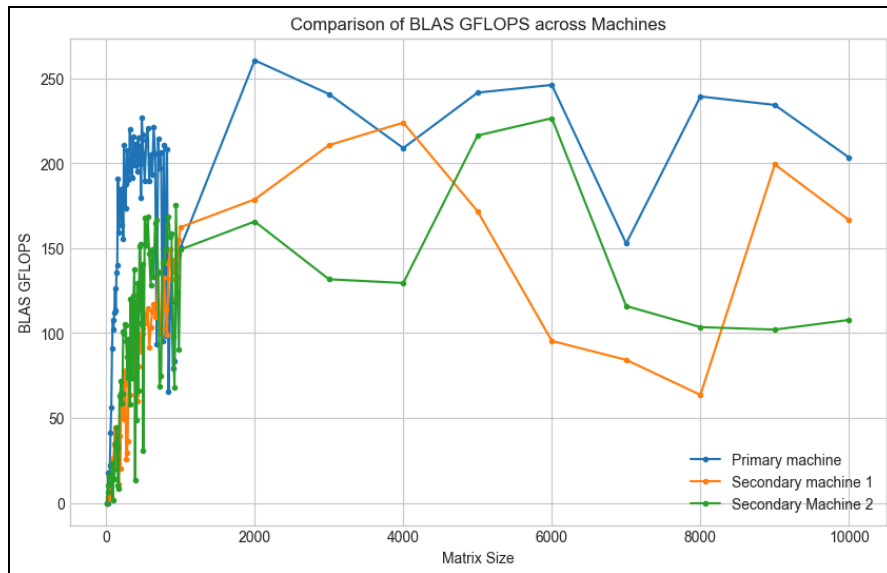
- **Lenovo (Secondary 1):** Slowest of all. The orange line spikes drastically at large matrix sizes (N>5000).
- **Mac M1 & M4: Faster.** Both Mac machines keep the execution time much lower and flatter than the Lenovo.
- **Reason:** This indicates Lenovo is hitting a **Memory Bottleneck** (L3 Cache contention). While the Ryzen has 8 cores, feeding them all data for large matrices causes thrashing in the shared 16MB L3 cache. The Macs' Unified Memory Architecture appears to handle concurrent memory access more gracefully for this specific naive algorithm.

BLAS Matrix Multiplication

- **Mac M1 (Primary):** Fastest of all. The blue line remains consistently lowest.
- **Mac M4 (Secondary 2):** Close Second. Tracks closely with M1 but slightly higher execution time in this specific run.
- **Lenovo (Secondary 1):** Slowest of all. The orange line shows significant spikes (e.g., at N=8000).
- **Reason:** The Apple Accelerate framework is highly tuned for the M-series chips, fully utilizing the NEON/AMX units. The generic BLAS on Lenovo (Ubuntu) is less optimized for the Ryzen architecture or is hitting thermal limits.

7.2.1 GFLOPS





Sequential GFLOPS

- **Lenovo:** Dominates, peaks at ~2.7 GFLOPS and maintains a lead until large matrix sizes.
- **Mac M1:** Slowest, drops rapidly to nearly 0 GFLOPS.
- **Reason:** The Ryzen 5800H's raw clock speed (up to 4.4GHz) and aggressive prefetching help it power through unoptimized, linear code faster than the 3.2GHz M1.

Threaded GFLOPS

- **Mac M1:** Highest Peak, hits ~6 GFLOPS for small matrices.
- **Lenovo:** Lowest Stability, starts high but crashes hard as matrix size increases, indicating severe thermal or cache throttling.
- **Reason:** The M1's Firestorm cores provide stable performance. The Lenovo likely suffers from **False Sharing** or cache contention as 16 logical threads fight for the same memory bus.

BLAS GFLOPS

- **Mac M1: Highest.** Peaks at ~260 GFLOPS and sustains high performance.
- **Lenovo: Variable.** Peaks around 220 GFLOPS but suffers massive drops (e.g., at N=8000, dropping to ~60 GFLOPS).
- **Mac M4: Surprising Result.** It peaks lower (~170-220 GFLOPS) than the M1 in this specific test.
- **Reason:**
 - **M1:** The Accelerate library is perfectly mature for M1.
 - **Lenovo:** High peak potential (AVX2 is powerful), but inconsistent. The drops are likely due to background processes or thermal throttling on the laptop form factor.
 - **M4:** The lower result is counter-intuitive. It could be due to the OS scheduler (running on E-cores instead of P-cores), library immaturity, or the test size not being large enough to trigger the M4's more powerful (but higher overhead) AMX units. There were also few processes running in the background which could have contributed to this difference.

8 Program Output

Below are the program output screenshots of three variations of the program.

8.1 Sequential vs BLAS

```
pooranits@MacBookAir GEMM-Code-Results % clang++ -O0 -std=c++17 Lab2_GEMM_MM_Sequential_BLAS.cpp -framework Accelerate -o Lab2_GEMM_MM_Sequential_BLAS
Lab2_GEMM_MM_Sequential_BLAS.cpp:112:5: warning: 'cblas_dgemm' is deprecated: first deprecated in macOS 13.3 - An updated CBLAS interface supporting ILP64 is available. Please compile with -DACCELERATE_NEW_LAPACK to access the new headers and -DACCELERATE_LAPACK_ILP64 for ILP64 support. [-Wdeprecated-declarations]
  112 |     cblas_dgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans,
      |             ^
/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/System/Library/Frameworks/vecLib.framework/Headers/cblas.h:647:6: note: 'cblas_dgemm' has been explicitly marked deprecated here
  647 | void cblas_dgemm(const enum CBLAS_ORDER __Order,
      |             ^
1 warning generated.
pooranits@MacBookAir GEMM-Code-Results % ./Lab2_GEMM_MM_Sequential_BLAS --range_start 10 --range_end 10000

----- Matrix Multiplication Stats -----
Matrix Size: 10x10
-----
Sequential Duration (ns): 19750
BLAS Duration (ns): 81917
-----
Seq vs BLAS Avg Error: 0.00000000001692
-----
Seq vs BLAS Max Error: 0.00000000007276
-----
Sequential GFLOPS: 0.10
BLAS GFLOPS: 0.02
-----
```

8.2 Parallel vs BLAS

```
pooranits@MacBookAir GEMM-Code-Results % clang++ -O0 -std=c++17 Lab2_GEMM_MM_Threaded_BLAS.cpp -framework Accelerate -o Lab2_GEMM_MM_Threaded_BLAS
Lab2_GEMM_MM_Threaded_BLAS.cpp:146:5: warning: 'cblas_dgemm' is deprecated: first deprecated in macOS 13.3 - An updated CBLAS interface supporting ILP64 is available. Please compile with -DACCELERATE_NEW_LAPACK to access the new headers and -DACCELERATE_LAPACK_ILP64 for ILP64 support. [-Wdeprecated-declarations]
  146 |     cblas_dgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans,
      |             ^
/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/System/Library/Frameworks/vecLib.framework/Headers/cblas.h:647:6: note: 'cblas_dgemm' has been explicitly marked deprecated here
  647 | void cblas_dgemm(const enum CBLAS_ORDER __Order,
      |             ^
1 warning generated.
pooranits@MacBookAir GEMM-Code-Results % ./Lab2_GEMM_MM_Threaded_BLAS --range_start 10 --range_end 10000 --threads_count 8

----- Matrix Multiplication Stats -----
Matrix Size: 10x10
-----
Threaded Duration (ns): 160959
BLAS Duration (ns): 39000
-----
Thread vs BLAS Avg Error: 0.00000000001692
-----
Thread vs BLAS Max Error: 0.00000000007276
-----
Threaded GFLOPS: 0.01
BLAS GFLOPS: 0.05
-----
```

8.3 Sequential, Parallel and BLAS

```
pooranits@MacBookAir GEMM-Code-Results % clang++ -O0 -std=c++17 Lab2_GEMM_MM_Multi_Consolidation.cpp -framework Accelerate -o Lab2_GEMM_MM_Multi_Consolidation
Lab2_GEMM_MM_Multi_Consolidation.cpp:181:5: warning: 'cblas_dgemm' is deprecated: first deprecated in macOS 13.3 - An updated CBLAS interface supporting ILP64 is available. Please compile with -DACCELERATE_NEW_LAPACK to access the new headers and -DACCELERATE_LAPACK_ILP64 for ILP64 support. [-Wdeprecated-declarations]
  181 |     cblas_dgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans,
      |             ^
/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/System/Library/Frameworks/vecLib.framework/Headers/cblas.h:647:6: note: 'cblas_dgemm' has been explicitly marked deprecated here
  647 | void cblas_dgemm(const enum CBLAS_ORDER __Order,
      |             ^
1 warning generated.
pooranits@MacBookAir GEMM-Code-Results % ./Lab2_GEMM_MM_Multi_Consolidation --range_start 10 --range_end 10000 --threads_count 8

----- Matrix Multiplication Stats -----
Matrix Size: 10x10
-----
Sequential Duration (ns): 10791
Threaded Duration (ns): 167958
BLAS Duration (ns): 51125
-----
Seq vs Thread Avg Error: 0.00000000000000
Seq vs BLAS Avg Error: 0.00000000001692
Thread vs BLAS Avg Error: 0.00000000001692
-----
Seq vs Thread Max Error: 0.00000000000000
Seq vs BLAS Max Error: 0.00000000007276
Thread vs BLAS Max Error: 0.00000000007276
-----
Sequential GFLOPS: 0.19
Threaded GFLOPS: 0.01
BLAS GFLOPS: 0.04
-----
```

9 GEMV comparison with GEMM

9.1 Similarities

- In both GEMV and GEMM operations, BLAS significantly outperformed sequential and multi-threading methods.
- The execution time for sequential exponentially increases for larger size matrices.

9.2 Differences

- In case of error rates of Sequential and Multi-threaded with BLAS, there is volatility for larger matrices. But for GEMV operation there is no volatility and there is a predictable gradual increase.
- The execution time for larger matrices is very high (>12 hours) for matrix multiplication, when compared to execution time (in minutes) of larger matrices for matrix vector multiplication.

10 Future Work

Below are some methods to improve the performance of the matrix multiplication.

- **Compiler Optimization:** The main problem is the use of the -O0 flag. This flag tells the compiler not to optimize anything. By changing this to -O3 or -Ofast, the compiler will be able to make some changes to the code. It will do things like auto-vectorization on its own. This means it will use the instructions that are built into our computers hardware like NEON on Mac or AVX, on Ryzen. When we do this our program will run a lot faster even 10 to 50 times faster. The best part is we do not have to change our source code at all.
- **Algorithmic Optimization:** The significant spikes in execution time observed as the matrix size (N) increases are attributed to Cache Thrashing. To mitigate this, Block Matrix Multiplication (Tiling) could be implemented. By partitioning the matrix into smaller tiles that fit entirely within the cache, data is kept hot near the processor, and the latency penalties associated with main memory access are avoided.
- **Memory Optimization:** In standard multiplication loops, Matrix A is accessed efficiently row by row, while Matrix B is accessed column by column. This "strided" access pattern is known to hurt performance because it does not make use of hardware prefetching mechanisms. To fix this, Matrix B should be transposed in memory before the computation loop. This restructuring ensures that both matrices are read sequentially (row-major), allowing the memory subsystem to fully take advantage of spatial locality.
- **Parallelism Strategy:** The instability seen in the threaded implementation at high N values is likely due to False Sharing. This happens when multiple threads try to modify variables on the same 64-byte cache line. To solve this, memory workloads need to be aligned. Each thread should be given a separate chunk of the result matrix that aligns with cache line boundaries. This prevents threads from invalidating shared cache lines and reduces congestion on the memory bus.

11 Conclusion

This report proves the notion that hardware optimizations are the key factor that determines the computational efficiency of matrix multiplication. On all the tested architectures, namely the Apple M1, M4, and AMD Ryzen 7 5800H, the BLAS implementation outperformed the naive sequential and multi-threaded approaches by a large margin, achieving a maximum of 260 GFLOPS on the Apple M1 processor via the NEON vectorization and efficient cache tiling strategy. Although the AMD Ryzen processor had high sequential performance due to the high clock frequency, the multi-threaded performance was severely affected by the L3 cache coherence issue, whereas the Apple M1 processor was able to maintain high throughput using the unified memory architecture. Finally, the exponential complexity of the naive sequential implementation proves the need to use optimized libraries to perform large-scale linear algebra computations efficiently.

To prove the correctness of the implementation, the analysis revealed that the tested architectures had varying behavior regarding the precision of the floating-point arithmetic results. On the Apple M1 processor, the results had a volatile behavior depending on the alignment of the matrices with the cache tiles, whereas the AMD system had a monotonic increase in the results. On the M4 processor, the results had the most stable behavior, possibly due to the improved cache hierarchy and optimized library functions. To improve the performance of the naive implementation to the level of the optimized implementation, future improvements can be made at the compiler level using auto-vectorization flags `-O3`, along with algorithmic improvements such as the block matrix multiplication method to overcome the problem of false sharing and cache thrashing.

References

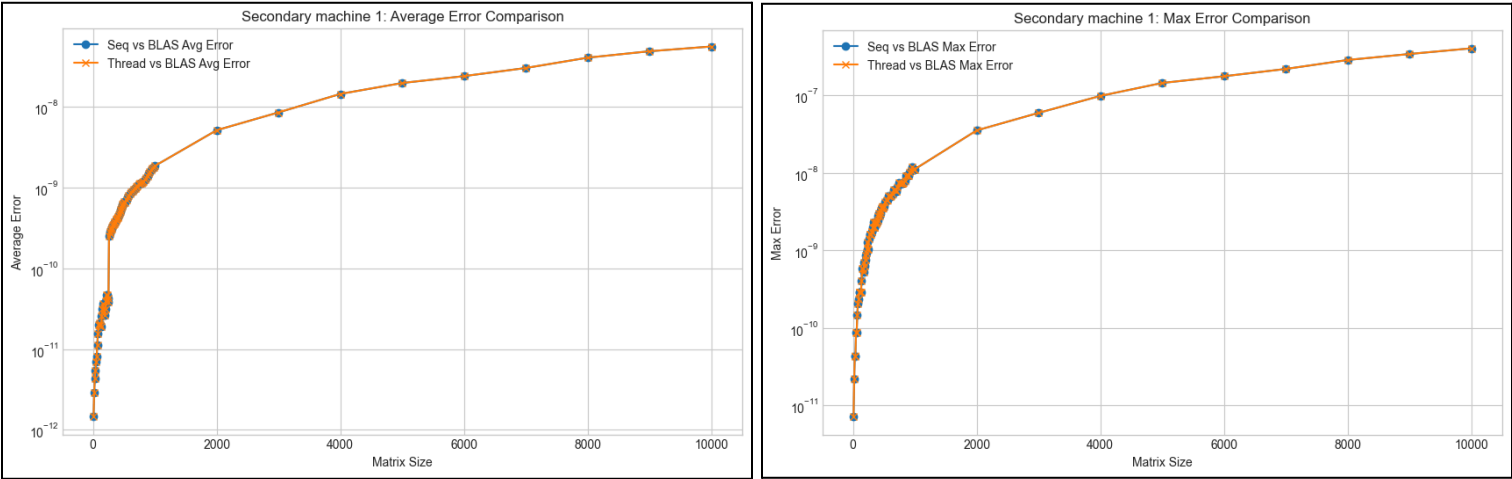
1. Dongarra, J. J., et al. (2002). An Updated Set of Basic Linear Algebra Subprograms (BLAS). *ACM Transactions on Mathematical Software*, 28(2), 135–151.
2. Goldberg, D. (1991). What Every Computer Scientist Should Know About Floating-Point Arithmetic. *ACM Computing Surveys*, 23(1), 5–48.
3. Golub, G. H., & Van Loan, C. F. (2013). *Matrix Computations* (4th ed.). Johns Hopkins University Press.
4. Hennessy, J. L., & Patterson, D. A. (2017). *Computer Architecture: A Quantitative Approach* (6th ed.). Morgan Kaufmann.
5. Apple Inc. (2024). *Accelerate Framework Documentation*. Apple Developer Archive.
6. Intel Corporation. (2024). *Intel 64 and IA-32 Architectures Optimization Reference Manual*
7. Class Materials & Slides

Appendix

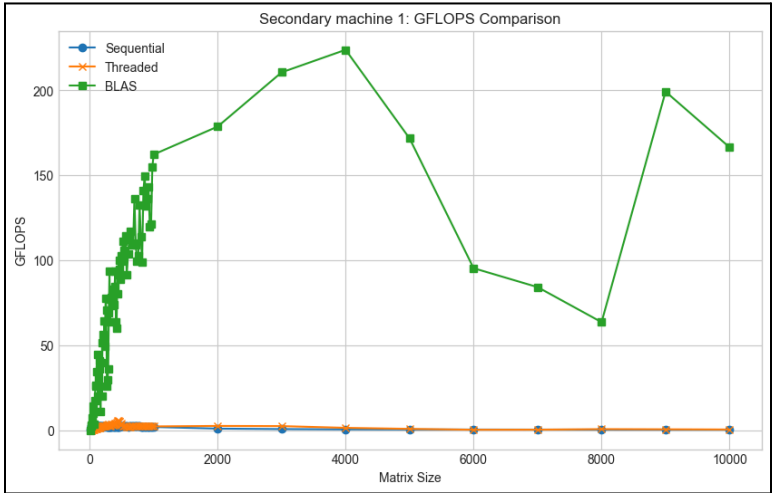
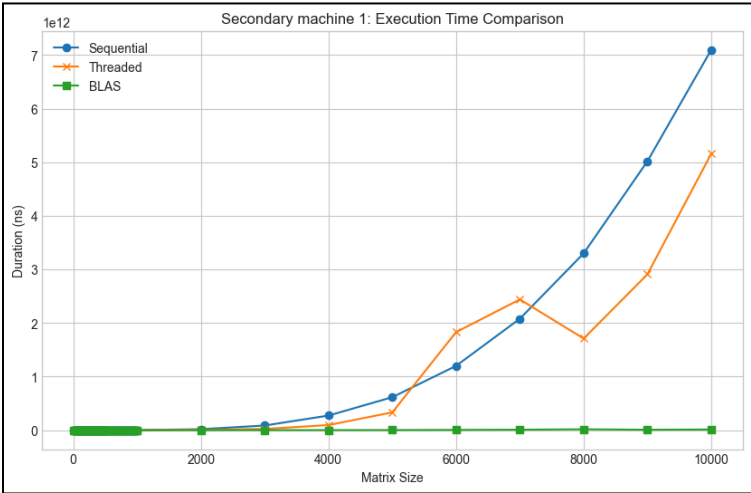
1 Experiment Results

1.1 Secondary Machine 1

1.1.1 Correctness comparison

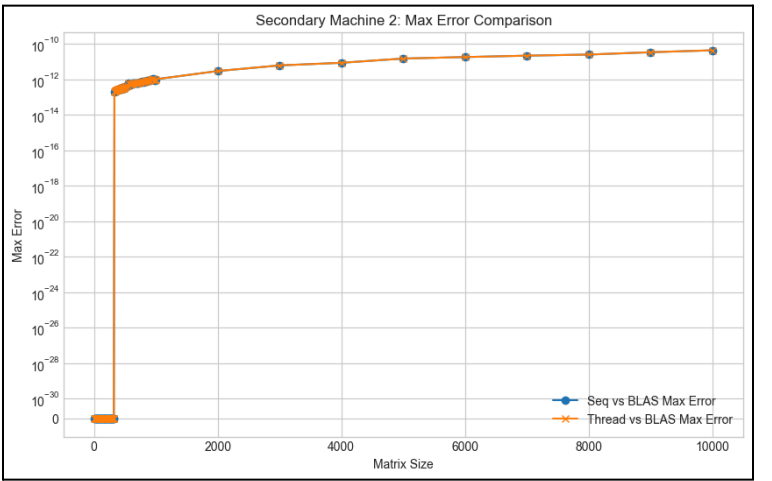
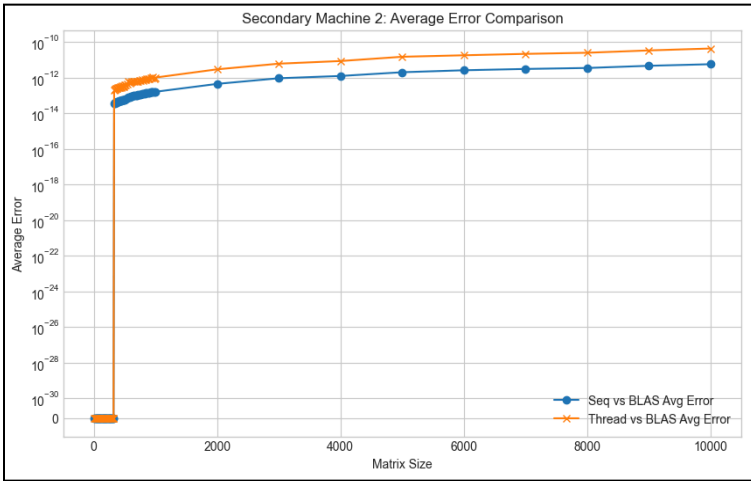


1.1.2 Execution Time comparison



1.2 Secondary Machine 2

1.2.1 Correctness comparison



1.2.2 Execution Time comparison

